

Associative Container

关联式容器

set

`set` 是关联容器，含有键值类型对象的已排序集，搜索、移除和插入拥有对数复杂度。`set` 内部通常采用 [红黑树](#) 实现。[平衡二叉树](#) 的特性使得 `set` 非常适合处理需要同时兼顾查找、插入与删除的情况。

和数学中的集合相似，`set` 中不会出现值相同的元素。如果有相同元素的集合，需要使用 `multiset`。`multiset` 的使用方法与 `set` 的使用方法基本相同。

插入与删除操作

- `insert(x)` 当容器中没有等价元素的时候，将元素 `x` 插入到 `set` 中。
- `erase(x)` 删除值为 `x` 的所有元素，返回删除元素的个数。
- `erase(pos)` 删除迭代器为 `pos` 的元素，要求迭代器必须合法。
- `erase(first, last)` 删除迭代器在 范围内的所有元素。
- `clear()` 清空 `set`。

▼ insert 函数的返回值

`insert` 函数的返回值类型为 `pair<LTiterator, bool>`，其中 `iterator` 是一个指向所插入元素（或者是指向等于所插入值的原本就在容器中的元素）的迭代器，而 `bool` 则代表元素是否插入成功，由于 `set` 中的元素具有唯一性质，所以如果在 `set` 中已有等值元素，则插入会失败，返回 `false`，否则插入成功，返回 `true`；`map` 中的 `insert` 也是如此。

迭代器

`set` 提供了以下几种迭代器：

1. `begin()/cbegin()`
返回指向首元素的迭代器，其中 `*begin = front`。
2. `end()/cend()`
返回指向数组尾端占位符的迭代器，注意是没有元素的。
3. `rbegin()/crbegin()`
返回指向逆向数组的首元素的逆向迭代器，可以理解为正向容器的末元素。
4. `rend()/crend()`
返回指向逆向数组末元素后一位置的迭代器，对应容器首的前一个位置，没有元素。

以上列出的迭代器中，含有字符 `c` 的为只读迭代器，你 cannot 通过只读迭代器去修改 `set` 中的元素的值。如果一个 `set` 本身就是只读的，那么它的一般迭代器和只读迭代器完全等价。只读迭代器自 C++11 开始支持。

查找操作

- `count(x)` 返回 `set` 内键为 `x` 的元素数量。
- `find(x)` 在 `set` 内存在键为 `x` 的元素时会返回该元素的迭代器，否则返回 `end()`。
- `lower_bound(x)` 返回指向首个不小于给定键的元素的迭代器。如果不存在这样的元素，返回 `end()`。
- `upper_bound(x)` 返回指向首个大于给定键的元素的迭代器。如果不存在这样的元素，返回 `end()`。
- `empty()` 返回容器是否为空。
- `size()` 返回容器内元素个数。

▼ lower_bound 和 upper_bound 的时间复杂度

set 自带的 lower_bound 和 upper_bound 的时间复杂度为 。

但使用 algorithm 库中的 lower_bound 和 upper_bound 函数对 set 中的元素进行查询，时间复杂度为 。

▼ nth_element 的时间复杂度

set 没有提供自带的 nth_element。使用 algorithm 库中的 nth_element 查找第 大的元素时间复杂度为 。

如果需要使用平衡二叉树所具备的 查找第 大元素的功能，需要自己手写平衡二叉树或权值线段树，或者选择使用 pb_ds 库中的平衡二叉树。

使用样例

set 在贪心中的使用

在贪心算法中经常会需要出现类似 找出并删除最小的大于等于某个值的元素。这种操作能轻松地通过 set 来完成。

```
1 // 现存可用的元素
2 set<int> available;
3 // 需要大于等于的值
4 int x;
5
6 // 查找最小的大于等于x的元素
7 set<int>::iterator it = available.lower_bound(x);
8 if (it == available.end()) {
9     // 不存在这样的元素，则进行相应操作.....
10 } else {
11     // 找到了这样的元素，将其从现存可用元素中移除
12     available.erase(it);
13     // 进行相应操作.....
14 }
```

map

map 是有序键值对容器，它的元素的键是唯一的。搜索、移除和插入操作拥有对数复杂度。map 通常实现为 [红黑树](#)。

设想如下场景：现在需要存储一些键值对，例如存储学生姓名对应的分数：Tom 0, Bob 100, Alan 100。但是由于数组下标只能为非负整数，所以无法用姓名作为下标来存储，这个时候最简单的办法就是使用 STL 中的 map。

map 重载了 operator[]，可以用任意定义了 operator < 的类型作为下标（在 map 中叫做 key，也就是索引）：

```
1 map<Key, T> yourMap;
```

其中，Key 是键的类型，T 是值的类型，下面是使用 map 的实例：

```
1 map<string, int> mp;
```

map 中不会存在键相同的元素，multimap 中允许多个元素拥有同一键。multimap 的使用方法与 map 的使用方法基本相同。

► Warning

插入与删除操作

- 可以直接通过下标访问来进行查询或插入操作。例如 mp["Alan"]=100。
- 通过向 map 中插入一个类型为 pair<L TKey, T> 的值可以达到插入元素的目的，例如

```
mp.insert(pair<LTstring,int>("Alan",100));;
```

- `erase(key)` 函数会删除键为 `key` 的所有元素。返回值为删除元素的数量。
- `erase(pos)`: 删除迭代器为 `pos` 的元素，要求迭代器必须合法。
- `erase(first,last)`: 删除迭代器在 范围内的所有元素。
- `clear()` 函数会清空整个容器。

▼ 下标访问中的注意事项

在利用下标访问 `map` 中的某个元素时，如果 `map` 中不存在相应键的元素，会自动在 `map` 中插入一个新元素，并将其值设置为默认值（对于整数，值为零；对于有默认构造函数的类型，会调用默认构造函数进行初始化）。

当下标访问操作过于频繁时，容器中会出现大量无意义元素，影响 `map` 的效率。因此一般情况下推荐使用 `find()` 函数来寻找特定键的元素。

查询操作

- `count(x)`: 返回容器内键为 `x` 的元素数量。复杂度为（关于容器大小对数复杂度，加上匹配个数）。
- `find(x)`: 若容器内存在键为 `x` 的元素，会返回该元素的迭代器；否则返回 `end()`。
- `lower_bound(x)`: 返回指向首个不小于给定键的元素的迭代器。
- `upper_bound(x)`: 返回指向首个大于给定键的元素的迭代器。若容器内所有元素均小于或等于给定键，返回 `end()`。
- `empty()`: 返回容器是否为空。
- `size()`: 返回容器内元素个数。

使用样例

`map` 用于存储复杂状态

在搜索中，我们有时需要存储一些较为复杂的状态（如坐标，无法离散化的数值，字符串等）以及与之有关的答案（如到达此状态的最小步数）。`map` 可以用来实现此功能。其中的键是状态，而值是与之相关的答案。下面的示例展示了如何使用 `map` 存储以 `string` 表示的状态。

```
1 // 存储状态与对应的答案
2 map<string, int> record;
3
4 // 新搜索到的状态与对应答案
5 string status;
6 int ans;
7 // 查找对应的状态是否出现过
8 map<string, int>::iterator it = record.find(status);
9 if (it == record.end()) {
10     // 尚未搜索过该状态，将其加入状态记录中
11     record[status] = ans;
12     // 进行相应操作.....
13 } else {
14     // 已经搜索过该状态，进行相应操作.....
15 }
```

遍历容器

可以利用迭代器来遍历关联式容器的所有元素。

```
1 set<int> s;
2 using si = set<int>::iterator;
3 for (si it = s.begin(); it != s.end(); it++) cout << *it << endl;
```

需要注意的是，对 `map` 的迭代器解引用后，得到的是类型为 `pair<Key, T>` 的键值对。

在 C++11 中，使用范围 `for` 循环会让代码简洁很多：

```
1 set<int> s;  
2 for (auto x : s) cout << x << endl;
```

对于任意关联式容器，使用迭代器遍历容器的时间复杂度均为 $O(n)$ 。

自定义比较方式

`set` 在默认情况下的比较函数为 `<`（如果是非内置类型需要 [重载 < 运算符](#)）。然而在某些特殊情况下，我们希望能自定义 `set` 内部的比较方式。

这时候可以通过传入自定义比较器来解决问题。

具体来说，我们需要定义一个类，并在这个类中 [重载 \(\) 运算符](#)。

例如，我们想要维护一个存储整数，且较大值靠前的 `set`，可以这样实现：

```
1 struct cmp {  
2     bool operator()(int a, int b) const { return a > b; }  
3 };  
4  
5 set<int, cmp> s;
```

对于其他关联式容器，可以用类似的方式实现自定义比较，这里不再赘述。

本页面最近更新：2024/10/9 22:38:42, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[cmpute](#), [ouuan](#), [StudyingFather](#), [1475505](#), [billchenchina](#), [DeterCider](#), [Enter-tainer](#), [Haohu Shen](#), [lr1d](#), [ksyx](#), [Marcythm](#), [mgt](#), [ouuan](#), [sbofgayschool](#), [shuzhouliu](#), [Tiphereth-A](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用